

Урок 5. Корутины. Suspend функции изнутри

20 декабря 2019

В этом уроке разбираем, как работает suspend функция изнутри. Как она может вернуть значение сразу или после асинхронной работы.

Этот раздел необязателен для прочтения и понимания дальнейших уроков. Он позволит заглянуть под капот suspend функции и понять, как реализована двухрежимность. Т.е. как она может вернуть значение сразу или после выполнения работы.

Подробно

Рассмотрим suspend функцию:

```
1 private suspend fun download(url: String): File {
2     return suspendCoroutine { continuation ->
3
4         val file = getFileFromCache(url)
5
6         if (file != null) {
7             continuation.resume(file)
8         } else {
9             networkService.download(url, object : NetworkService.Callback {
10                 override fun onSuccess(result: File) {
11                     continuation.resume(result)
12                 }
13
14                 override fun onFailure(error: Exception) {
15                     continuation.resumeWithException(error)
16                 }
17             })
18         }
19     }
20 }
21 }
```

Она либо сразу возвращает значение из кэша, либо асинхронно скачивает файл.

Как и любой Kotlin код, эта функция будет трансформирована в Java. Упрощенный результат выглядит так:

```
1 Object download(String url, Continuation continuation) {
2
3     File file = getFileFromCache(url);
4
5     if (file != null) {
6         continuation.resume(file);
7     } else {
8         networkService.download(url, new NetworkService.Callback() {
9             @Override
10             public void onSuccess(File result) {
11                 continuation.resume(result);
12             }
13
14             @Override
15             public void onFailure(Exception error) {
16                 continuation.resumeWithException(error);
17             }
18         });
19     }
20 }
```

Код практически не изменился. Только вместо котлиновского suspendCoroutine мы имеем входной параметр Continuation. Когда Continuation вызывает эту функцию, он передает туда себя.

Функция возвращает Object. Тут есть два варианта. Она должна вернуть результат работы (файл), если он был передан в continuation.resume сразу из кэша (синхронный режим). Или она должна вернуть COROUTINE_SUSPENDED, если в continuation.resume сразу ничего не было передано (асинхронный режим).

Для обеспечения такой логики используется SafeContinuation обертка для Continuation. Я убрал его из выше рассмотренной упрощенной версии. Давайте вернем:

```

1  Object download(String url, Continuation continuation) {
2
3      SafeContinuation safeContinuation = new SafeContinuation(continuation);
4
5      File file = getFileFromCache(url);
6
7      if (file != null) {
8          safeContinuation.resume(file);
9      } else {
10         networkService.download(url, new NetworkService.Callback() {
11             @Override
12             public void onSuccess(File result) {
13                 safeContinuation.resume(result);
14             }
15
16             @Override
17             public void onFailure(Exception error) {
18                 safeContinuation.resumeWithException(error);
19             }
20         });
21     }
22
23     return safeContinuation.getOrThrow();
24 }

```

У `SafeContinuation` есть методы для приема результата (`resume`) или ошибки (`resumeWithException`), и метод для получения значения (`getOrThrow`). А в конструктор он получает `continuation`, который вызывал `suspend` функцию.

Давайте обратим внимание на порядок вызова методов `SafeContinuation.resume*` и `SafeContinuation.getOrThrow`. Этот порядок различен для синхронного и асинхронного режимов. Это очень важно, потому что от этого порядка зависит внутренняя логика работы `SafeContinuation`. Т.е. `resume` методы проверяют, был ли вызван `getOrThrow`, а `getOrThrow` проверяет были ли вызваны `resume*` методы. И в зависимости от результатов этих проверок выполняется разная логика.

Все так сложно потому, что мы используем метод `SafeContinuation.resume` как в случае, когда результат готов сразу, так и в случае, когда результат получен после асинхронной работы. И этот метод `resume` может быть вызван как до, так и после `getOrThrow`.

Рассмотрим синхронный сценарий. У нас есть значение в кэше (`file != null`). Значит, сразу будет вызван `safeContinuation.resume` метод и ему будет передан файл. Этот метод проверит (внутри `safeContinuation`), вызывался ли уже `safeContinuation.getOrThrow`. Выяснит, что пока не вызывался. Это говорит о том, что сценарий синхронный. В этом случае файл просто сохраняется внутри `SafeContinuation` и не передается в оригинальный `Continuation`.

Далее будет вызван метод `getOrThrow`. Он проверит, был ли вызов `resume` метода с передачей файла. Выяснит, что был. Для него это также сигнал, что сценарий синхронный. И он просто вернет сохраненный файл.

В итоге `suspend` функция синхронно вернет файл и `Continuation` перейдет в следующую ветку свитча.

В асинхронном сценарии все немного сложнее и порядок вызова методов будет другой. Кэш ничего не вернет, поэтому метод `resume` сразу вызван не будет. Вместо этого будет запущен асинхронный `networkService.download`, и не дожидаясь его результатов будет вызван `getOrThrow`.

`getOrThrow` проверит, вызывался ли `resume`. Выяснит, что нет. Это значит, что сразу получить значение не удалось, стартовала асинхронная работа, и результат будет позже. Вместо результата метод `getOrThrow` вернет `COROUTINE_SUSPENDED`. И `suspend` функция, соответственно, вернет это же значение. `Continuation` получит это значение и пока что закончит свою работу, ожидая когда его снова вызовут.

Когда `networkService.download` отработает, он вызовет колбэк, в котором будет вызван `resume` метод с передачей ему файла. Этот метод проверит, был ли вызван `getOrThrow`. Выяснит, что да. Значит сценарий асинхронный, и `Continuation`, который вызывал `suspend` функцию, ждет результатов.

`SafeContinuation` внутри себя вызовет метод `resume` у оригинального `Continuation` и передаст ему файл. Это в итоге приведет к вызову метода `invokeSuspend`.

Это были успешные сценарии. Давайте для полноты картины рассмотрим сценарии, когда мы вызываем `resumeWithException`. В примере выше я вызывал его только для асинхронного сценария. Но ничто не мешает вызывать его сразу, если логика в этом нуждается.

Рассмотрим синхронный сценарий. Например, кэш уже пытался скачать файл, выяснил, что его там нет и сохранил ошибку. Мы сразу вызываем `resumeWithException` метод и передаем ему ошибку. Метод `resumeWithException` проверит, вызывался ли `getOrThrow`. Выяснит, что пока не вызывался. Это говорит о том, что сценарий синхронный. В этом случае ошибка просто сохраняется внутри `SafeContinuation` и не передается в оригинальный `Continuation`.

Далее будет вызван метод `getOrThrow`. Он проверит, был ли вызов какого-либо `resume` метода. Выяснит, что был вызван `resumeWithException`. Для него это также сигнал, что сценарий синхронный. Он выбросит исключение, которое пришло в `resumeWithException`.

Это приведет к тому, что в недрах `Continuation` сработает `try-catch`, который поймает ошибку и корутина будет завершена с этой ошибкой.

В асинхронном сценарии будет запущен асинхронный `networkService.download`, и не дожидаясь его результатов будет вызван `getOrThrow`.

`getOrThrow` проверит, вызывался ли какой-то из `resume` методов. Выяснит, что нет. Это значит, что стартовала асинхронная работа, и результат будет позже. Вместо результата метод `getOrThrow` вернет `COROUTINE_SUSPENDED`. И `suspend` функция, соответственно, вернет это же значение. `Continuation` получит это значение и пока что закончит свою работу, ожидая когда его снова вызовут.

Позже метод `networkService.download` вызовет колбэк, в котором будет вызван `resumeWithException` метод с передачей ему ошибки. Этот метод проверит, был ли вызван `getOrThrow`. Выяснит, что да. Значит сценарий асинхронный, и `Continuation`, который вызывал `suspend` функцию, ждет результатов.

`SafeContinuation` внутри себя вызовет метод `resume` у оригинального `Continuation` и передаст ему ошибку. `Continuation` определит, что пришла ошибка, выбросит исключение, сам же его поймает и корутина будет завершена с этой ошибкой.

Я надеюсь, мне удалось объяснить тему `Continuation`. Она была одной из самых сложных для понимания в корутинах. Дальше будет полегче, поинтереснее и ближе к практике.

Кратко

Рассмотрим suspend функцию

```
1 private suspend fun download(url: String): File {
2     return suspendCoroutine { continuation ->
3
4         val file = getFileFromCache(url)
5
6         if (file != null) {
7             continuation.resume(file)
8         } else {
9             networkService.download(url, object : NetworkService.Callback {
10                 override fun onSuccess(result: File) {
11                     continuation.resume(result)
12                 }
13
14                 override fun onFailure(error: Exception) {
15                     continuation.resumeWithException(error)
16                 }
17             })
18         }
19     }
20 }
21 }
```

Она либо сразу возвращает значение из кэша, либо асинхронно скачивает файл.

Как и любой Kotlin код, эта функция будет трансформирована в Java. Результат выглядит так:

```
1 Object download(String url, Continuation continuation) {
2
3     SafeContinuation safeContinuation = new SafeContinuation(continuation);
4
5     File file = getFileFromCache(url);
6
7     if (file != null) {
8         safeContinuation.resume(file);
9     } else {
10         networkService.download(url, new NetworkService.Callback() {
11             @Override
12             public void onSuccess(File result) {
13                 safeContinuation.resume(result);
14             }
15
16             @Override
17             public void onFailure(Exception error) {
18                 safeContinuation.resumeWithException(error);
19             }
20         });
21     }
22
23     return safeContinuation.getOrThrow();
24 }
```

Как вы помните по прошлым урокам, suspend функция при преобразовании получает дополнительный параметр Continuation. Он и используется в нашем коде. Никакого котлиновского suspendCoroutine здесь уже нет.

Функция возвращает Object. Она должна вернуть результат работы (файл), если он был передан в continuation сразу из кэша (синхронный режим). Или она должна вернуть COROUTINE_SUSPENDED, если в continuation сразу ничего не было передано (асинхронный режим). Для обеспечения этой двухрежимной логики используется SafeContinuation-обертка для нашего Continuation.

У SafeContinuation есть две входные точки для получения результата (resume) или ошибки (resumeWithException), и выходная точка для получения итогового результата (getOrThrow) suspend функции. Также он хранит в себе continuation, который вызывал suspend функцию.

Логика работы `SafeContinuation` зависит от того, какой метод был вызван первым: `resume*` или `getOrThrow`. Все так сложно потому, что мы используем метод `SafeContinuation.resume` как в случае синхронного, так и асинхронного сценария. И он может быть вызван как до, так и после `getOrThrow`.

В случае синхронного сценария сначала будет вызван `resume` метод, который просто сохранит внутри `SafeContinuation` переданный ему файл. Далее будет вызван метод `getOrThrow`, который вернет сохраненный файл. В итоге `suspend` функция синхронно вернет файл и `Continuation` перейдет в следующую ветку свитча.

В асинхронном сценарии сначала будет вызван `getOrThrow`. Вместо результата он вернет `COROUTINE_SUSPENDED`, и `suspend` функция, соответственно, вернет это же значение. Когда `networkService.download` отработает, он передаст файл в `resume` метод, а он уже передаст этот файл в метод `resume` у оригинального `Continuation`.

Давайте для полноты картины рассмотрим сценарии, когда мы вызываем `resumeWithException`. В примере выше я вызывал его только для асинхронного сценария. Но ничто не мешает вызывать его сразу, если логика в этом нуждается.

В случае синхронного сценария сначала будет вызван `resumeWithException` метод, который просто сохранит внутри `SafeContinuation` переданную ему ошибку. Далее будет вызван метод `getOrThrow`, который увидит сохраненную ошибку и выбросит исключение. Это приведет к тому, что в недрах `Continuation` сработает `try-catch`, который поймает ошибку и корутина будет завершена с этой ошибкой.

В асинхронном сценарии сначала будет вызван `getOrThrow`. Вместо результата он вернет `COROUTINE_SUSPENDED`, и `suspend` функция, соответственно, вернет это же значение. Когда `networkService.download` отработает, он передаст ошибку в `resumeWithException` метод, а он уже передаст эту ошибку в метод `resume` у оригинального `Continuation`. `Continuation` определит, что пришла ошибка, выбросит исключение, сам же его поймает и корутина будет завершена с этой ошибкой.